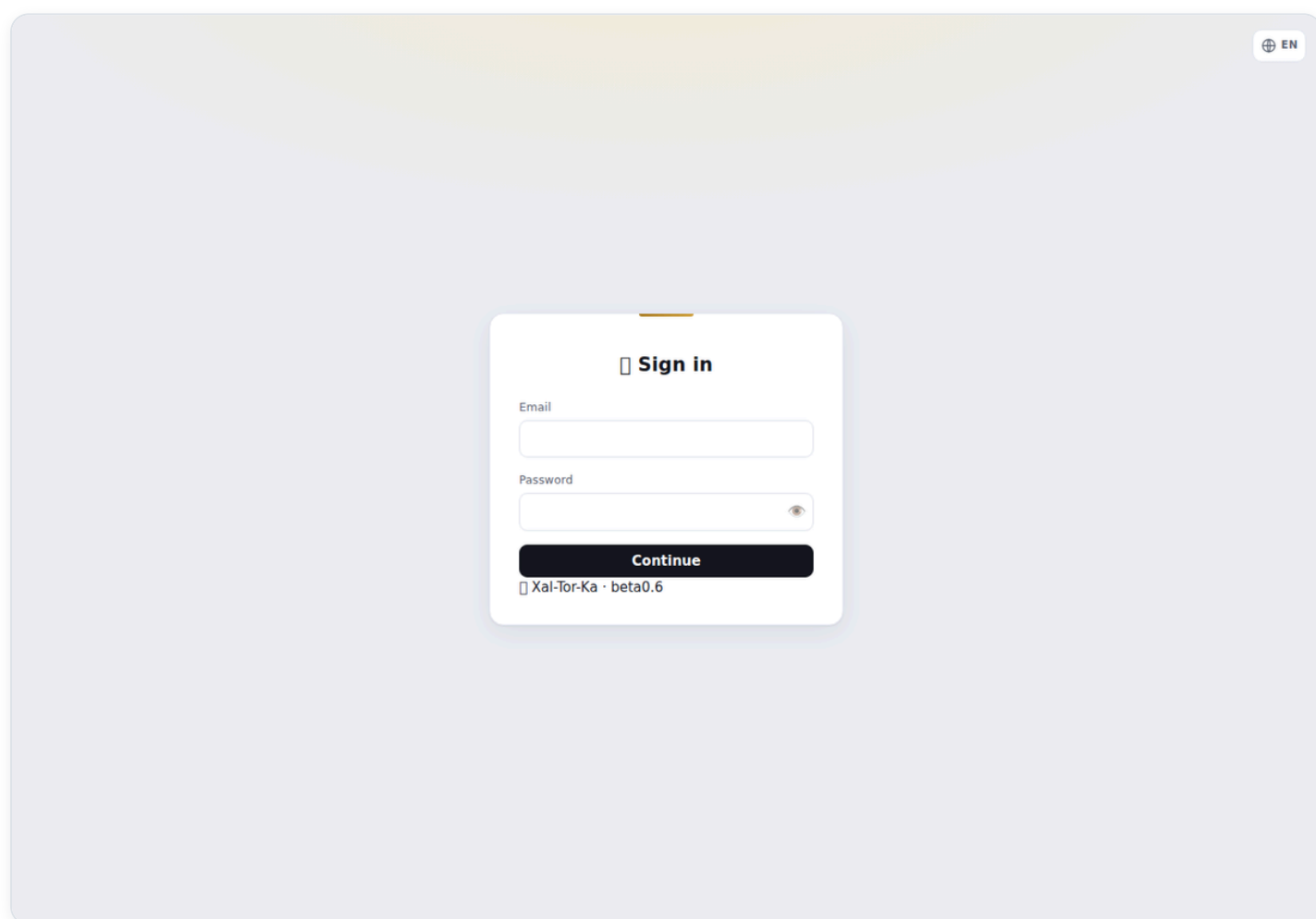


Xal-Tor-Ka — guida pratica (come si fa)

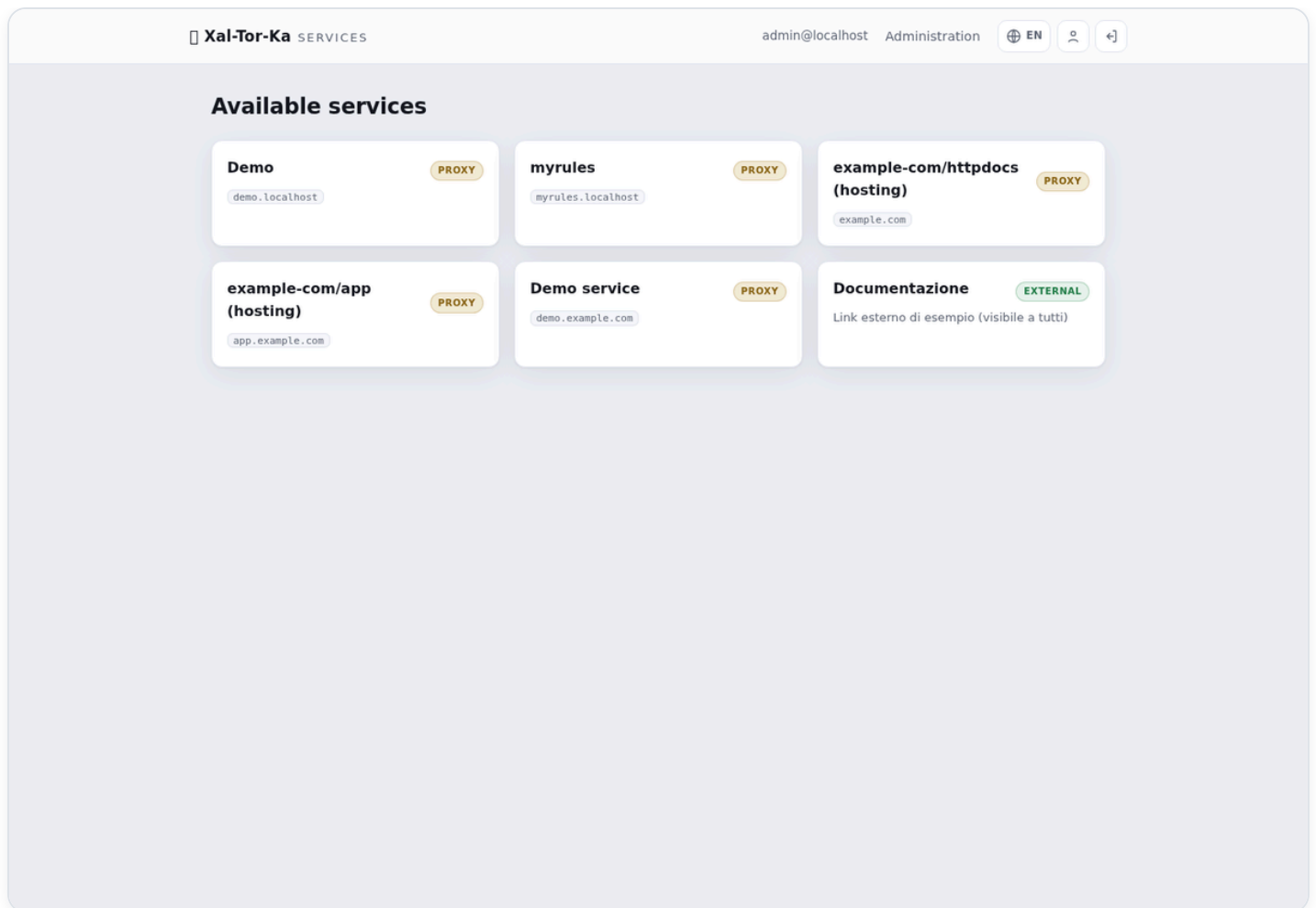
Ricettario operativo: le cose che si fanno più spesso, passo-passo. Presuppone un'istanza già installata (vedi [INSTALL.md](#)) e un utente admin. Per il *perché* e il valore vedi [xaltorka-overview.md](#). Screenshot da installazione dimostrativa.

Accesso

Vai all'URL del gateway e fai login. Gli utenti locali usano email + password (+ codice TOTP se il 2FA è attivo); in alternativa un pulsante per ogni provider OIDC configurato. L'area di amministrazione è accessibile solo dagli **IP in whitelist**.



Dopo il login vedi il **catalogo dei servizi** che ti sono visibili. La voce **Administration** in alto porta al pannello di gestione.



Ricetta 1 — Pubblicare un servizio esistente (reverse-proxy)

Hai già un servizio che gira (un container, un backend interno) e vuoi metterlo dietro il gate.

1. **Administration → Services → Add backend.**
2. Compila: - **Host pubblico:** il dominio da cui si accede (es. `app.example.com`). - **Upstream:** l'indirizzo interno del servizio (es. `http://mio-servizio:8080`; anche un servizio sulla rete Docker interna, non solo sulle reti del computer). - **Regola:** `public` (aperto), `authenticated` (richiede login), `whitelist` (solo IP consentiti).
3. Salva. Il backend compare nell'elenco e il routing NGINX si aggiorna.

Xal-Tor-KaADMINISTRATIONbeta0.6

Services

DockerProvidersUsersMonitoringTLSHostingDashboard

EN

Reverse-proxy services

Services from config.json are read-only (infrastructure). Those added here live in services.json.

SERVICE	HOST / UPSTREAM	RULE	IP ALLOW	
Demo	demo.localhost http://demo:80	PUBLIC		editManage SSLdisabledelete
myrules-builder-backend-1 OFF	myrules-builder-backend-1.localhost http://host.docker.internal:8765	PUBLIC		editManage SSLenabledelete
myrules	myrules.localhost http://host.docker.internal:5173	AUTHENTICATED		editManage SSLdisabledelete
example-com/httpdocs (hosting)	example.com http://example-com.site:8080	PUBLIC		editManage SSLdisabledelete
example-com/app (hosting)	app.example.com http://example-com-app.site:8080	PUBLIC		editManage SSLdisabledelete
Demo service	demo.example.com http://demo:80	AUTHENTICATED		editManage SSLdisabledelete

Add proxied service

idnamehostpathrule

app.example.com

/

whitelist

upstreampublic urlwwwIP allow (CIDR, opt.)

http://10.0.0.5:8080https://app.example.c

☐ also www.<host>

203.0.113.0/24

add

public: open to everyone. authenticated: users with a valid session (2FA). whitelist: only explicitly authorized users (implies authentication); administrators always have access (override).

External links

Tiles in the dashboard, not proxied.

NAME	URL	VISIBILITY	
Documentazione Link esterno di esempio (visibile a tutti)	https://example.com/docs	PUBLIC	disabledelete

Add link

idnamepublic urldescription

https://...

☐ public

add

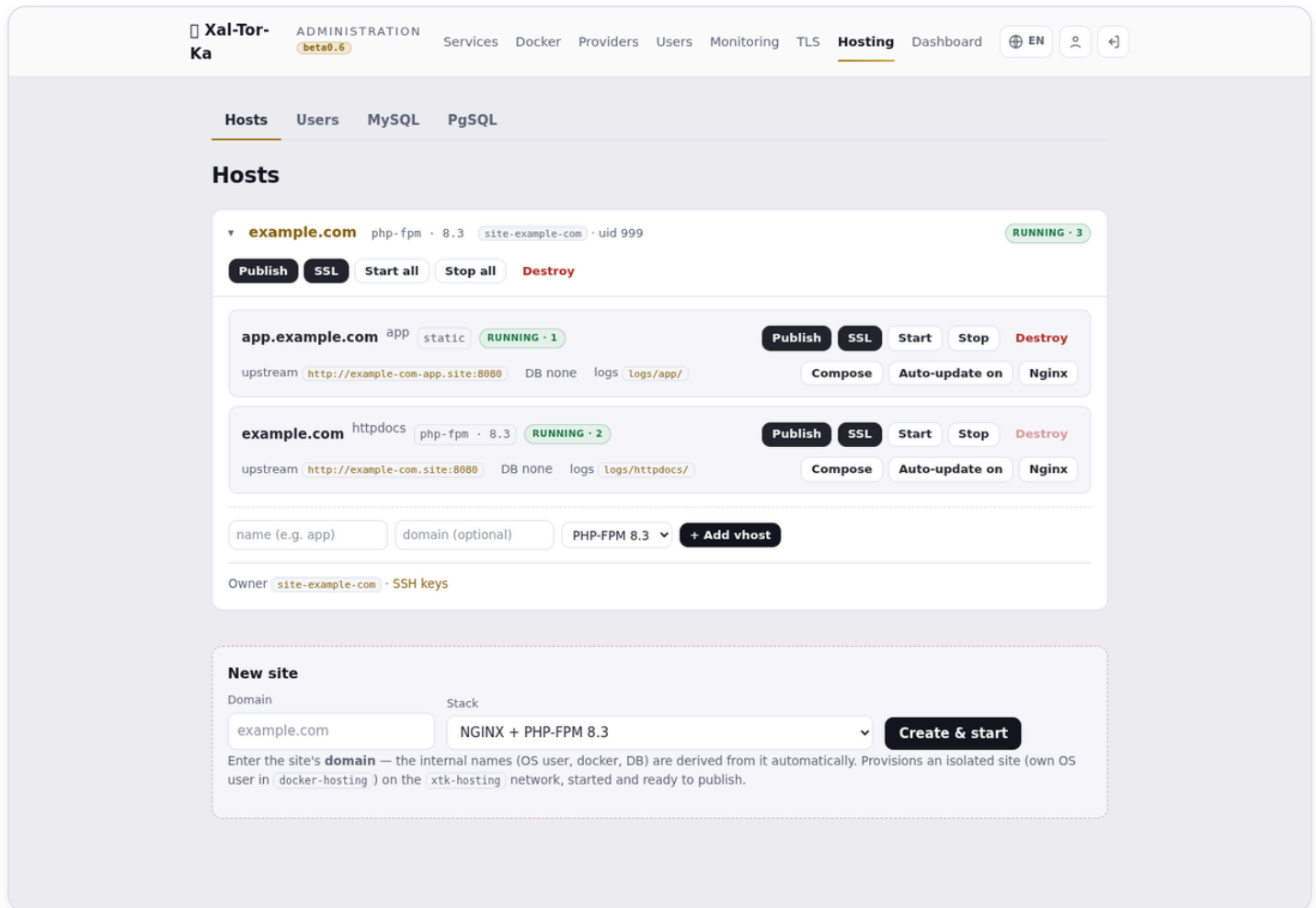
Poi dai un certificato all'host (Ricetta 4) e punta il DNS del dominio al gateway.

Ricetta 2 — Creare un sito in hosting (Docker)

Xal-Tor-Ka provisiona un sito isolato da zero: utente di sistema dedicato + Docker, sulla rete `xtk-hosting`.

1. **Administration** → **Hosting** → riquadro **New site** in fondo.

2. Inserisci il **dominio** (es. `example.com`) e scegli lo **stack**: NGINX + PHP-FPM (8.1/8.2/8.3), con MySQL o PostgreSQL condiviso, statico, o *custom* (scrivi tu il compose).
3. **Create & start**. Nomi interni (utente OS, container, DB) derivati dal dominio.



Il sito appare come scheda con il suo vhost `httpdocs` già avviato. Da qui gestisci Start/Stop, Compose, Nginx, e l'accesso.

Ricetta 3 — Aggiungere un sottodominio (vhost) e pubblicarlo

Un sito può avere più vhost (es. `app.`, `api.`), ognuno nella sua Docker.

1. Nella scheda del sito, riga **+ Add vhost**: nome (es. `app`), dominio pubblico opzionale (es. `app.example.com`), stack.
2. Il vhost parte con il suo container.
3. **Publish**: apri il dialog del vhost, conferma l'**host pubblico** e la **regola** → crea il backend reverse-proxy verso quel vhost.

4. (Poi) **SSL** → emetti il certificato (Ricetta 4).

Il pulsante **Publish** diventa verde quando il servizio è attivo, rosso se disabilitato, nero se non ancora pubblicato. Idem **SSL** per lo stato del certificato.

Ricetta 4 — Emettere un certificato TLS

Administration → TLS. La pagina elenca gli **host serviti** (quelli con un backend). I sottodomini compaiono annidati sotto il dominio padre.

- **issue LE** (Let's Encrypt): per host **pubblici** che risolvono al gateway con la porta 80 raggiungibile. La spunta **www.** aggiunge anche `www.<host>`.
- **self-signed**: per host **LAN/dev** senza DNS pubblico; usa la **CA interna** (scaricabile in fondo alla pagina, da installare sui client per farla fidare).

Xal-Tor-Ka ADMINISTRATION
beta0.6

Services Docker Providers Users Monitoring TLS Hosting Dashboard

EN

Certificates

Issue and renew TLS certificates for the served hosts. Let's Encrypt (ACME) needs the host to resolve publicly to this gateway with port 80 reachable; use the internal CA for LAN/dev hosts without public DNS.

HOST	SOURCE	EXPIRY	STATUS				
demo.localhost	—	—	MISSING	☐ www.	issue LE	self-signed	delete
myrules.localhost	—	—	MISSING	☐ www.	issue LE	self-signed	delete
example.com	internal CA	2028-10-20	VALID	☐ www.	issue LE	self-signed	delete
↳ app.example.com	internal CA	2028-10-20	VALID	☐ www.	issue LE	self-signed	delete
↳ demo.example.com	—	—	MISSING	☐ www.	issue LE	self-signed	delete

Internal CA

Download internal CA

Install this CA on client devices to trust the self-signed certificates.

ACME account email: — — set `tls.acme.email` in `config.json` (Let's Encrypt expiry notices)

Nota: il certificato **segue il backend**. Se un host non compare in TLS, prima pubblicalo (Ricetta 1/3). Per l'ACME il container può anche essere spento: la challenge la risponde il gateway.

Ricetta 5 — Aggiungere un provider OIDC (SSO)

Administration → Providers → Add.

1. Scegli un **preset** (Google, Microsoft) o *custom*.
2. Compila **id** (mnemonico), **name** (etichetta sul pulsante), **issuer**, **client_id**, **client_secret**, e spunta **enabled**.
3. Salva. Comparirà un pulsante di login dedicato nella pagina di accesso.

Xal-Tor-KaADMINISTRATIONbeta0.6Services Docker Providers Users Monitoring TLS Hosting DashboardEN

Authentication providers

External OIDC login providers, managed at runtime. Local password login is always available.

ID	NAME	TYPE	ENABLED	ISSUER	SECRET
local CONFIG		LOCAL	ON	—	— config.json — edit the file + rebuild
google CONFIG	Google	OIDC	OFF	https://accounts.google.com	— config.json — edit the file + rebuild
microsoft CONFIG	Microsoft	OIDC	OFF	https://login.microsoftonline.com/<TENANT_ID>/v2.0	— config.json — edit the file + rebuild

Add provider

presetcustom / generic OIDC

prefills the issuer for common providers

idgoogle

short identifier, used in the login URL /auth/<id>/... (not editable later)

nameGoogle

label shown on the login button

issuerhttps://accounts.google.com

OIDC issuer URL (discovery base), e.g. https://accounts.google.com

client id1234....apps.googleusercontent.com

OAuth client id issued by the provider

client secret

OAuth client secret; write-only, never shown back — leave empty to keep the current one

enabled☐

to log in, discovery against the issuer must succeed (use test)

add

Ricetta 6 — LDAP / Active Directory

Per autenticare con gli account di dominio: configura la fonte LDAP (bind LDAPS/StartTLS, template del DN, base DN). Il login locale prova prima le credenziali locali, poi fa fallback su LDAP — un bind riuscito completa l'autenticazione. Gli account di **Active Directory** funzionano via bind LDAPS al domain controller (gli account **locali Windows** no: il SAM non è esposto a un gate Linux). *Dettagli in `docs/next-gen-auth-sources.md`.*

Ricetta 7 — Database condivisi e Adminer

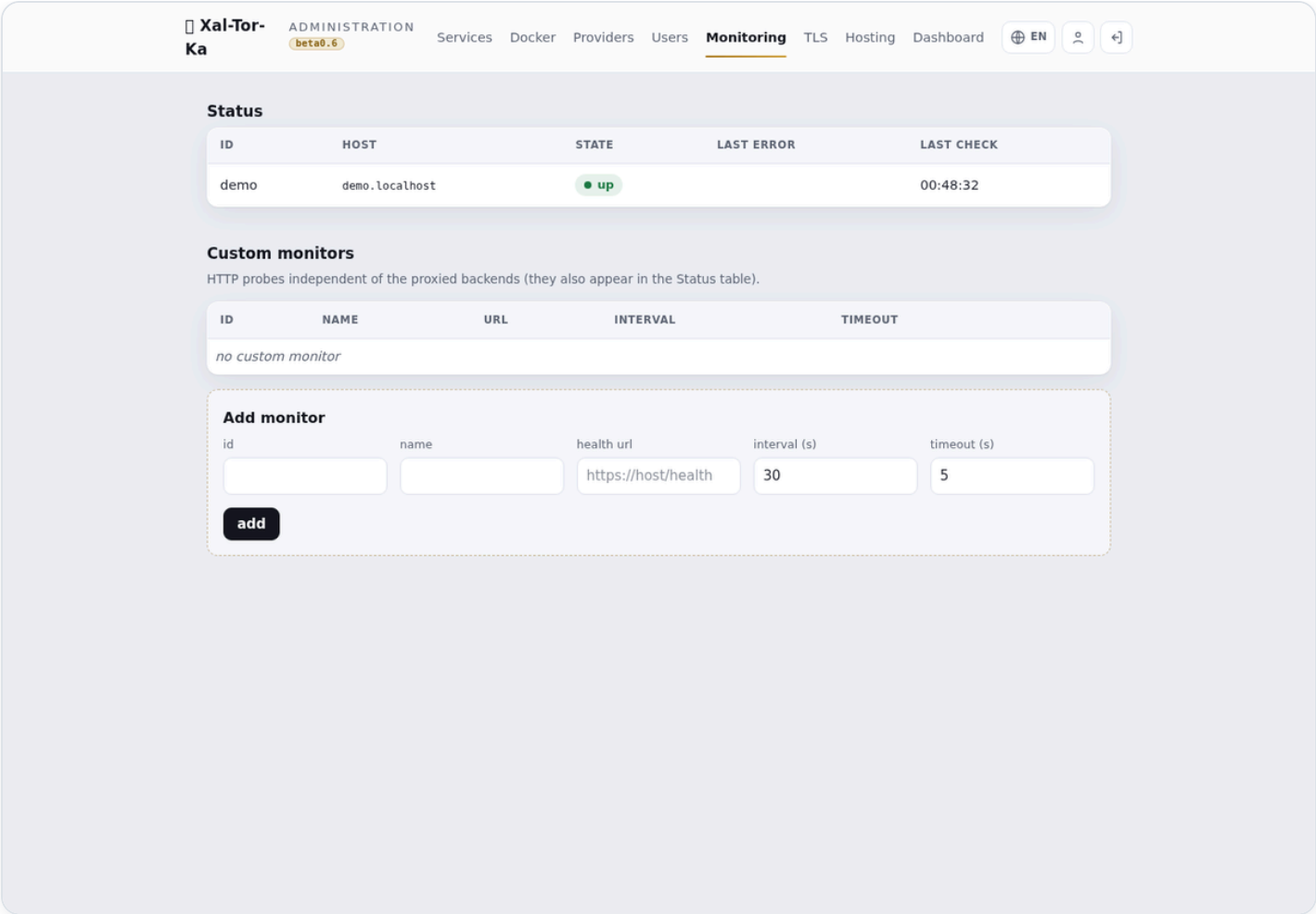
Negli stack hosting puoi attaccare un **MySQL** o **PgSQL condiviso**: le tab **MySQL/PgSQL** in Hosting gestiscono le istanze condivise; a ogni sito che lo richiede viene assegnato un DB con utente isolato (nomi/credenziali generati). Un **Adminer** effimero permette di ispezionare i DB.

Ricetta 8 — Accesso SFTP/SSH al sito

Ogni sito ha un utente di sistema con **chroot** per SFTP/SCP (porta dedicata). Dalla scheda del sito → **SSH keys** (o **Owner**): genera/gestisci le **chiavi pubbliche** dell'utente (usabili al posto della password), con download del file combo. Le chiavi si incollano/modificano dal pannello.

Ricetta 9 — Blindare l'admin e le notifiche remote

- **Whitelist IP admin**: imposta gli IP/CIDR autorizzati all'area di amministrazione (il tuo IP pubblico/VPN in produzione). Tutto il resto resta fuori dall'admin.
- **Notifiche/controllo remoto** (opzionale): configura un bot **Telegram** e/o **SMTP/IMAP** per ricevere log di sistema a distanza e mandare **comandi vettati** (allow-list mittenti + comandi; email firmate DKIM). Utile come log-system remoto e per operazioni base senza aprire il pannello.



Riferimento rapido

Voglio...	Vai a
Mettere auth/HTTPS davanti a un servizio	Services → Add backend, poi TLS
Creare un sito nuovo	Hosting → New site
Aggiungere un sottodominio	Hosting → scheda sito → + Add vhost → Publish
Un certificato	TLS → issue LE / self-signed
SSO aziendale	Providers (OIDC) / config LDAP
Accesso ai file del sito	Hosting → scheda sito → SSH keys (SFTP)
Chi può entrare nell'admin	Whitelist IP admin

Xal-Tor-Ka è software di SFS.it. Screenshot da installazione dimostrativa con dati di esempio.